

UNIVERSIDAD DON BOSCO
TÉCNICO EN INGENIERÍA EN COMPUTACIÓN
DESARROLLO DE APLICACIONES CON SOFTWARE PROPIETARIO



TAREA DE INVESTIGACIÓN

Grupo de Teórico: 03

Fecha: 10/09/2026

Docente: Ing. Juan Carlos Menjivar Ramírez

Presentado por:

No.	APELLIDO	NOMBRE	CARNET
1	Ruiz Hernández	Edgar Antonio	RH201851
2	Henriquez Vasquez	Axel Francisco	HV230423
3	Varela Linares	Marjorie Daniela	VL261354
4	Ramirez Torres	Eduardo Alfredo	RT240549
5	Azucena Ayala	Carlos Josue	AA260854
6	Ayala Palacios	Marcos Ezequiel	AP260351
7	Henriquez Ponce	Diego Noel	HP160046

1. Fundamentos de Aplicaciones de Escritorio en .NET

1.1 Windows Forms: Definición y Comparativa

Windows Forms (WinForms) es el marco clásico de .NET para crear interfaces gráficas de usuario en escritorio, operando como una envoltura (*wrapper*) sobre la API nativa de Windows (User32 y GDI+).

- **Frente a WPF:** WPF utiliza un motor de renderizado vectorial basado en DirectX (acelerado por GPU) y separa el diseño (XAML) de la lógica (MVVM), mientras que WinForms depende de la CPU y mezcla el diseño con código imperativo en C#.
- **Frente a .NET MAUI:** MAUI es multiplataforma (ejecuta un solo código en Windows, macOS, Android e iOS), mientras que WinForms está restringido exclusivamente a Windows.

1.2 Arquitectura de Eventos y Controles

La arquitectura de WinForms se fundamenta en el *Message Loop* del sistema operativo y en el modelo de eventos/delegados de .NET.

- **Controles (System.Windows.Forms.Control):** Cada elemento visual (botones, cajas de texto) hereda de esta clase y encapsula un identificador nativo (*HWND*).
- **Message Loop:** Al iniciar la aplicación, un hilo de interfaz entra en un ciclo constante que escucha las acciones del usuario (teclado, ratón) enviadas por el sistema operativo.
- **Eventos y Delegados:** La aplicación traduce dichos mensajes a eventos estándar mediante la firma *EventHandler*, asociando el método manejador en el *Code-Behind*.

1.3 WinForms vs. Desarrollo Web (ASP.NET)

Ventajas de WinForms:

- **Acceso nativo:** Comunicación directa sin restricciones con hardware (puertos serie, escáneres) y sistema de archivos local.
- **Stateful:** La aplicación reside en la memoria RAM, evitando la pérdida de estado propia de HTTP.
- **Offline:** Funciona sin dependencia de conexión a internet o servidores web.

Limitaciones de WinForms:

- **Distribución:** Requiere instalación o actualización del ejecutable en cada máquina individual.
- **Portabilidad:** Limitado exclusivamente al entorno de escritorio Windows.
- **Seguridad:** La conexión directa a bases de datos (ADO.NET) puede exponer credenciales en la máquina del cliente, a diferencia del modelo servidor-cliente de ASP.NET Core.

2. Consumo de Servicios Web REST desde C#

¿Qué es una API REST?

Una API REST es una interfaz de programación de aplicaciones que se ajusta a los principios de diseño del estilo arquitectónico REST, utilizado para conectar sistemas y exponer flujos de integración entre aplicaciones. El cliente y el servidor son independientes entre sí, la comunicación es sin estado, y se realiza mediante verbos HTTP (GET, POST, PUT, DELETE) sobre recursos identificados por una URL, devolviendo generalmente los datos en formato JSON.

A. La clase HttpClient y buenas prácticas de uso

- **Reutilización de instancias:** Crear un nuevo HttpClient en cada petición agota los sockets disponibles del sistema; se recomienda una única instancia estática o usar IHttpClientFactory para centralizar la gestión de conexiones.
- **Tiempos de espera y asincronía:** Configurar un Timeout evita que la aplicación se bloquee esperando respuesta, y usar async/await impide que la interfaz se congele durante la petición.
- **Manejo de errores:** Verificar el código de estado de la respuesta (EnsureSuccessStatusCode) permite distinguir entre una petición exitosa y un fallo del servidor o del recurso.

B. Serialización y deserialización con System.Text.Json

- **Qué es:** Librería nativa de .NET para convertir datos entre JSON y objetos C#, sin dependencias externas.
- **Deserialización:** La respuesta JSON de la API se transforma en objetos o listas de objetos C# fuertemente tipados para trabajar con ellos en el código.
- **Serialización:** De forma inversa, un objeto C# se convierte en JSON antes de enviarlo en el cuerpo de una petición (por ejemplo, al crear o actualizar un recurso).

3. Programación Asíncrona en C#

A-) Introducción a async/await

La programación asíncrona permite que una aplicación realice operaciones que toman tiempo, como consultar un servicio web, sin bloquear la ejecución del programa. En C#, las palabras clave `async` y `await` facilitan este proceso. `async` identifica un método asíncrono y `await` permite esperar el resultado de una operación sin bloquear el hilo.

Esto es especialmente útil en aplicaciones con interfaces gráficas, ya que evita que la ventana se quede congelada mientras se espera una respuesta de internet.

B-) Diferencias entre programación síncrona y asíncrona

- **Síncrona:** Las operaciones se ejecutan una después de otra y una tarea debe terminar antes de continuar.
- **Asíncrona:** Permite esperar operaciones que toman tiempo sin bloquear el programa, por lo que la aplicación puede seguir respondiendo.

Por ejemplo, al descargar información de un servicio web, la programación asíncrona permite que la interfaz siga funcionando mientras llega la respuesta.

C-) Buenas prácticas y errores comunes

Al utilizar async/await se recomienda:

- Utilizar await en lugar de .wait() o .Result, ya que estos pueden bloquear la aplicación y provocar problemas en interfaces gráficas.
- Nombrar los métodos asíncronos terminándolos en Async, por ejemplo, CargarDatosAsync().
- Usar Task o Task<T> para los métodos asíncronos.
- Utilizar async void principalmente en controladores de eventos, como el evento de un botón.
- Manejar los posibles errores con try-catch.

En interfaces gráficas, el objetivo principal es evitar bloquear el hilo de la interfaz para que el usuario pueda seguir interactuando con la aplicación.

4. Integración de Datos: Persistencia Local y API Externa

¿Qué es la integración de API?

La integración de API se refiere al uso de interfaces de programación de aplicaciones (API) para exponer flujos de integración y conectar aplicaciones, sistemas y flujos de trabajo de software empresarial para el intercambio de datos y servicios.

A. Cómo combinar datos obtenidos de una API con almacenamiento local (archivo o base de datos)

- ❖ **Arquitectura de Sincronización:** Consiste en consumir la API externa (vía HttpClient), deserializar la respuesta JSON a objetos en memoria (List<T>) y luego consolidar/persistir esos objetos en una base de datos local (SQLite, SQL Server) o un archivo plano (JSON, CSV).

- ❖ **Patrón de Abstracción y Desacoplamiento:** Como se menciona en los conceptos de integración de API, aislar la fuente remota de la capa local mediante interfaces y modelos intermedios permite que la aplicación continúe operando y mostrando datos incluso ante fallos de red o desconexión (modo *offline-first*).
- ❖ **Integración de Fuentes y Mapeo:** Cuando los datos provienen de múltiples endpoints o formatos, se recomienda procesar los archivos o respuestas (p. ej., matrices JSON o archivos CSV) e importarlos mediante colecciones o tareas en segundo plano hacia la base de datos o almacenamiento local, asegurando la consistencia e integridad.

B. Casos de Uso en la Industria: Aplicaciones de escritorio que sincronizan información con la nube

- ❖ **Sistemas POS (Puntos de Venta) y Gestión de Inventarios:** Registran ventas localmente en una base de datos liviana o archivo para operar sin interrupciones. Al detectar conexión a Internet, sincronizan el stock con el ERP/CRM en la nube a través de peticiones HTTP/REST.
- ❖ **Sistemas Enterprise / Gestión de Activos (Ej. IBM Maximo Manage):** Aplicaciones de escritorio o móviles que exportan e importan grandes volúmenes de datos mediante archivos planos (CSV, JSON) en procesos asíncronos o tareas en segundo plano (*cron jobs*), permitiendo la manipulación de información local y su posterior consolidación en la nube o AWS S3.
- ❖ **Herramientas de Análisis y Consultas Frecuentes (Caso de la Guía Práctica):** Aplicaciones de escritorio en Windows Forms que guardan listas de favoritos o historiales en un archivo local (p. ej., favoritos.csv o favoritos.json) utilizando List<T>, permitiendo al usuario revisar la información de la API de forma persistente aunque la red falle.

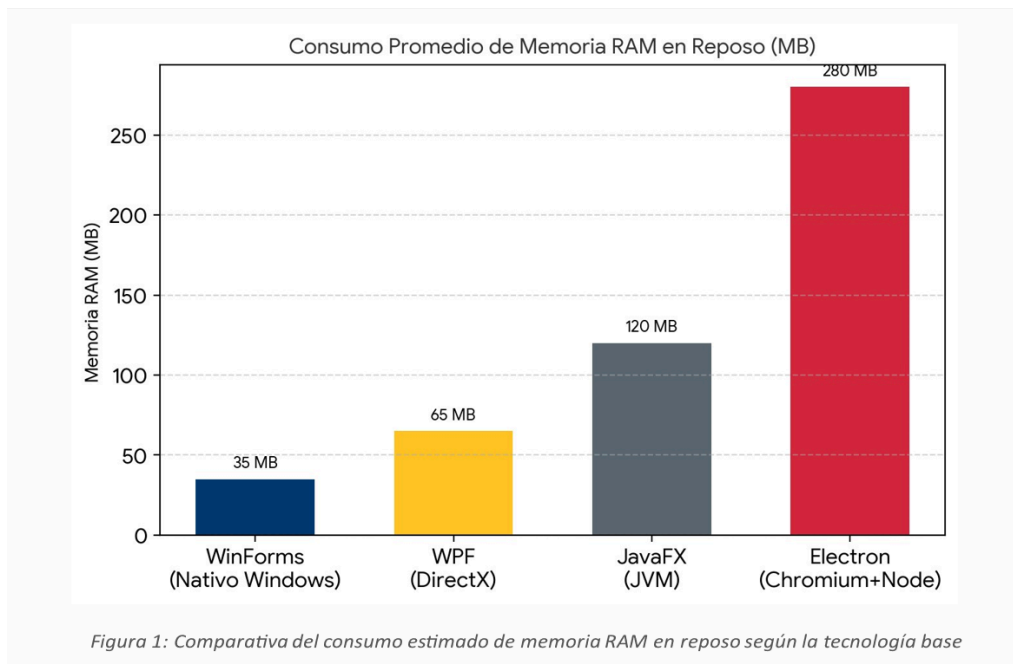
5. Comparación con Otras Tecnologías de Escritorio y Multiplataforma

- Breve comparación con .NET MAUI, Electron y JavaFX.
- Criterios para decidir cuándo conviene usar Windows Forms frente a otras alternativas.

WinForms / WPF: Alta integración con el ecosistema de Windows. Excelente rendimiento en RAM.

.NET MAUI: Solución multiplataforma de Microsoft (Windows, macOS, Android, iOS), abstrayendo controles nativos.

Electron: Utiliza tecnologías web (HTML, CSS, JavaScript a través de Chromium y Node.js). Facilita el desarrollo, pero incrementa dramáticamente el consumo de memoria y tamaño de instalación [3].



6. Bibliografía

- [1] H. M. Y. Al-Barhamtoshy, S. M. Al-Ghamdi, and others, "Comparative Study on WPF and Win-Forms Frameworks," *Int. J. Adv. Res. Comput. Commun. Eng.*, vol. 11, no. 7, pp. 388-393, Jul. 2022. Disponible: <https://ijarcce.com/wp-content/uploads/2022/07/IJARCCE.2022.11774.pdf>
- [2] A. Alkhars, "Cross-Platform Desktop Development (JavaFX vs. Electron)," Tesis de Grado, Umeå University, Umeå, Suecia, Mar. 2017. Disponible: <https://www.diva-portal.org/smash/record.jsf?pid=diva2:1081105>
- [3] J. Arteaga, "Propuesta arquitectónica para la migración de un sistema monolítico a un sistema distribuido utilizando domain driven design y microservicios: caso de estudio," Repositorio Institucional Universidad Don Bosco, San Salvador, El Salvador. [En línea]. Disponible: <https://rd.udb.edu.sv/bitstreams/cc39266c-342f-4d5a-9532-229e9cf08a8c/download>.
- [4] N. García and J. Linares, "Manual para la implementación de CMMI a nivel de capacidad 2 con SCRUM en el área de desarrollo de software," Repositorio Institucional Universidad Don Bosco, San Salvador, El Salvador, 2020. [En línea]. Disponible: <https://rd.udb.edu.sv/items/ff9f0c7a-8f6e-4050-a227-ff1f914f8d3f>.
- [5] Redalyc, "Journal de Ciencias Aplicadas e Investigación Tecnológica", Red de Revistas Científicas de América Latina y el Caribe, España y Portugal, 2022. [En línea]. Disponible: <https://www.redalyc.org/journal/474/47471710009/>
- [6] Microsoft, "Asynchronous programming scenarios," Microsoft Learn. [En línea]. Disponible: <https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/async-scenarios>
- [7] Microsoft, "Escenarios de programación asincrónica," Microsoft Learn. [En línea]. Disponible: <https://learn.microsoft.com/es-es/dotnet/csharp/asynchronous-programming/async-scenarios>
- [8] Microsoft, "Tipos de valor devuelto asincrónicos (C#)," Microsoft Learn. [En línea]. Disponible: <https://learn.microsoft.com/es-es/dotnet/csharp/asynchronous-programming/async-return-types>